

PROPOSTA TÉCNICA

Plano Técnico de Modelagem e API Curricular

Banco de dados + Módulo NestJS para disponibilizar dados do Plano Curricular por API

Base de referência	Tabela curricular enviada na conversa, com foco na página 321
Documento fonte	Plano Curricular da Rede Municipal de Educação - Biguaçu/SC
URL informada	https://www.bigua.sc.gov.br/uploads/sites/277/2023/06/2495359_PLANO_CURRICULAR_DA_REDE_MUNICIPAL.pdf
Área inicial	Linguagens
Componente inicial	Língua Portuguesa
Ano inicial	1º ano
Data da proposta	30/06/2026

Objetivo do plano

Converter a estrutura da tabela curricular em um banco relacional normalizado e em um módulo NestJS capaz de fornecer os dados por API REST, com filtros por ano, área, componente, trimestre, unidade temática, objeto de conhecimento e habilidade.

Trecho visual de referência

Imagem enviada na conversa e usada como base para identificar a hierarquia da tabela: ano, área, componente, trimestre, unidade temática, objeto de conhecimento e habilidades.

1º ANO – Área Linguagem: LÍNGUA PORTUGUESA		
UNIDADE TEMÁTICA	OBJETOS DE CONHECIMENTO / CONTEÚDOS	HABILIDADES
1º, 2º 3º TRIMESTRES		
Leitura /escuta (compartilhada e autônoma)	Reconstrução das condições de produção e recepção de textos	(EF15LP01) Identificar a função social de textos que circulam em campos da vida social dos quais participa cotidianamente (a casa, a rua, a comunidade, a escola) e nas mídias impressa, de massa e digital, reconhecendo para que foram produzidos, onde circulam, quem os produziu e a quem se destinam.
	Estratégia de leitura	(I) (EF15LP02) Estabelecer expectativas em relação ao texto que vai ler (pressuposições antecipadoras dos sentidos, da forma e da função social do texto), apoiando-se em seus conhecimentos prévios sobre as condições de produção e recepção desse texto, o gênero, o suporte e o universo temático, bem como sobre saliências textuais, recursos gráficos, imagens, dados da própria obra (índice, prefácio etc.), confirmando antecipações e inferências realizadas antes e durante a leitura de textos, checando a adequação das hipóteses realizadas. (II) (EF15LP03) Localizar informações explícitas em textos. (II) (EF15LP04) Identificar o efeito de sentido produzido pelo uso de recursos expressivos gráfico-visuais em textos multissemióticos.
	Leitura de imagens em narrativas visuais	(II) (EF15LP14) Construir o sentido de histórias em quadrinhos e tirinhas, relacionando imagens e palavras e interpretando recursos gráficos (tipos de balões, de letras, onomatopéias).
	Formação do leitor literário	(II) (EF15LP15) Reconhecer que os textos literários fazem parte do mundo do imaginário e apresentam uma dimensão lúdica, de encantamento, valorizando-os, em sua diversidade cultural, como patrimônio artístico da humanidade.
	Leitura colaborativa e autônoma	(II) (EF15LP16) Ler e compreender, em colaboração com os colegas e com a ajuda do professor e, mais tarde, de maneira autônoma, textos narrativos de maior porte como contos (populares, de fadas, acumulativos, de assombração etc.) e crônicas.
	Apreciação estética/Estilo	(II) (EF15LP17) Apreciar poemas visuais e concretos, observando efeitos de sentido criados pelo formato do texto na página, distribuição e diagramação das letras, pelas ilustrações e por outros efeitos visuais.
	Formação do leitor literário/Leitura Multissemiótica	(II) (EF15LP18) Relacionar texto com ilustrações e outros recursos gráficos.
	Planejamento de texto	(II) (EF15LP05) Planejar, com a ajuda do professor, o texto que será produzido, considerando a situação comunicativa, os interlocutores (quem escreve/para quem escreve); a finalidade ou o propósito (escrever para quê); a circulação (onde o texto vai circular); o suporte (qual é o portador do texto); a linguagem, organização e forma do texto e seu tema, pesquisando em meios impressos ou digitais, sempre que for preciso, informações necessárias à produção do texto, organizando em tópicos os dados e as fontes pesquisadas.

Sumário executivo

A tabela curricular deve ser tratada como uma hierarquia de informações, e não como uma planilha única. A estrutura recomendada separa documento, ano escolar, área do conhecimento, componente curricular, trimestres, unidades temáticas, objetos de conhecimento e habilidades. Essa separação evita duplicidade, facilita filtros e permite reaproveitar habilidades em diferentes itens curriculares.

Resultado esperado

Um backend NestJS com Prisma e PostgreSQL, organizado por módulo curricular, expondo endpoints REST para consulta dos itens curriculares e preparado para receber a importação completa do PDF.

Escopo funcional

- Cadastrar e consultar planos curriculares por documento, ano, área e componente.
- Consultar itens curriculares com filtros combináveis.
- Buscar rapidamente por código de habilidade, como EF15LP01 ou EF15LP02.
- Preservar página de origem e ordem dos itens, facilitando conferência com o documento original.
- Permitir expansão futura para outros anos, áreas, componentes e versões do currículo.

Escopo técnico

Camada	Proposta
API	NestJS com controllers, services e DTOs de consulta.
Banco	PostgreSQL, por ser adequado para dados relacionais e consultas filtradas.
ORM	Prisma, com schema tipado e migrations.
Importação	Arquivo JSON intermediário gerado a partir da tabela/PDF e validado antes do seed.
Documentação	Swagger/OpenAPI para facilitar testes e integração com frontend.

1. Leitura da tabela original

A tabela apresenta células mescladas e uma lógica de repetição visual. Por exemplo, a unidade temática pode aparecer uma vez na lateral e valer para várias linhas de objetos de conhecimento. Por isso, o importador deve “preencher” essas células mescladas durante a normalização, repetindo o valor correto para cada item curricular.

Coluna na tabela	Entidade sugerida	Observação
1º ANO - Área Linguagem: Língua Portuguesa	CurriculumPlan	Combina ano escolar, área e componente.
1º, 2º e 3º trimestres	Term + CurriculumItemTerm	Um item pode pertencer a um ou mais trimestres.
Unidade temática	ThematicUnit	Exemplo: Leitura/escuta compartilhada e autônoma.
Objetos de conhecimento / conteúdos	KnowledgeObject	Exemplo: Estratégia de leitura.
Habilidades	Skill + CurriculumItemSkill	Cada habilidade possui código e texto.
Página fonte	CurriculumItem.sourcePage	Ajuda auditoria e conferência manual.

Hierarquia conceitual

```

Documento curricular
├─ Plano curricular: 1º ano + Linguagens + Língua Portuguesa
│   └─ Trimestres: 1, 2 e 3
│       └─ Itens curriculares
│           ├── Unidade temática
│           ├── Objeto de conhecimento / conteúdo
│           └─ Habilidades
│               ├── Código: EF15LP01
│               └─ Texto da habilidade
  
```

Exemplo de transformação para JSON

```

{
  "ano": 1,
  "area": "Linguagens",
  "componente": "Língua Portuguesa",
  "trimestres": [1, 2, 3],
  "unidadeTematica": "Leitura/escuta compartilhada e autônoma",
  "objetoConhecimento": "Estratégia de leitura",
  "habilidades": [
    {
      "codigo": "EF15LP02",
      "texto": "Texto integral da habilidade extraído da tabela."
    }
  ],
  "paginaFonte": 321,
  "ordem": 2
}
  
```

2. Modelagem de banco de dados

A modelagem abaixo mantém os dados normalizados. O item curricular é a entidade central de consulta, conectando plano, trimestre, unidade temática, objeto de conhecimento e habilidades.

Entidade	Responsabilidade	Campos principais
CurriculumDocument	Identifica o documento fonte.	title, municipality, state, sourceUrl, version
SchoolYear	Representa ano/série.	code, stage, gradeNumber, label
KnowledgeArea	Representa a área do conhecimento.	name, slug
Subject	Representa o componente curricular.	areald, name, slug
CurriculumPlan	Combina documento + ano + componente.	documentId, schoolYearId, subjectId
Term	Representa trimestre.	number, label
ThematicUnit	Unidade temática dentro do plano.	planId, name, slug, position
KnowledgeObject	Objeto/conteúdo dentro da unidade.	thematicUnitId, name, position
Skill	Habilidade curricular/BNCC.	code, text, subjectId
CurriculumItem	Linha lógica da tabela.	planId, thematicUnitId, knowledgeObjectId, sourcePage

Relações principais

```

CurriculumDocument 1 — N CurriculumPlan
SchoolYear          1 — N CurriculumPlan
KnowledgeArea       1 — N Subject
Subject             1 — N CurriculumPlan
CurriculumPlan     1 — N ThematicUnit
ThematicUnit        1 — N KnowledgeObject
CurriculumPlan     1 — N CurriculumItem
CurriculumItem     N — N Term
CurriculumItem     N — N Skill
  
```

Decisões importantes de modelagem

- Habilidade deve ter código único, pois o mesmo código pode aparecer em mais de um item ou ano.
- Trimestre deve ser uma relação N:N, pois uma linha pode valer para 1º, 2º e 3º trimestres ao mesmo tempo.
- Unidade temática e objeto de conhecimento ficam separados para permitir filtros e agrupamentos no frontend.
- sourcePage e position preservam rastreabilidade com o documento original.
- Slugs devem ser salvos para facilitar URLs e filtros: lingua-portuguesa, linguagens, estrategia-de-leitura.

3. Prisma schema base

O schema abaixo é uma versão inicial e enxuta. Ele pode ser copiado para o arquivo `prisma/schema.prisma` e depois expandido conforme a importação completa do documento avançar.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum EducationStage {
  EDUCACAO_INFANTIL
  ENSINO_FUNDAMENTAL
}

model CurriculumDocument {
  id          String    @id @default(uuid())
  title       String
  municipality String?
  state       String?   @db.VarChar(2)
  sourceUrl   String?   @db.Text
  version     String?
  plans       CurriculumPlan[]
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@map("curriculum_documents")
}

model SchoolYear {
  id          String    @id @default(uuid())
  code        String    @unique
  stage       EducationStage
  gradeNumber Int?
  label       String
  plans       CurriculumPlan[]

  @@map("school_years")
}

model KnowledgeArea {
  id      String    @id @default(uuid())
  name    String
  slug    String    @unique
  subjects Subject[]

  @@map("knowledge_areas")
}

model Subject {
  id      String    @id @default(uuid())
  areaId  String
  area    KnowledgeArea @relation(fields: [areaId], references: [id])
  name    String
  slug    String
  plans   CurriculumPlan[]
  skills  Skill[]

  @@unique([areaId, slug])
  @@map("subjects")
}

model CurriculumPlan {
  id          String    @id @default(uuid())
  documentId  String
  schoolYearId String
  subjectId   String
  document    CurriculumDocument @relation(fields: [documentId], references: [id])
  schoolYear  SchoolYear         @relation(fields: [schoolYearId], references: [id])
  subject     Subject            @relation(fields: [subjectId], references: [id])
  title       String?
}
```

```

pageStart      Int?
pageEnd        Int?
thematicUnits  ThematicUnit[]
knowledgeObjects KnowledgeObject[]
items          CurriculumItem[]

@@unique([documentId, schoolYearId, subjectId])
@@index([schoolYearId, subjectId])
@@map("curriculum_plans")
}

model Term {
  id      String @id @default(uuid())
  number  Int    @unique
  label   String
  items   CurriculumItemTerm[]

  @@map("terms")
}

model ThematicUnit {
  id      String @id @default(uuid())
  planId  String
  plan    CurriculumPlan @relation(fields: [planId], references: [id])
  name    String
  slug    String
  position Int
  objects KnowledgeObject[]
  items   CurriculumItem[]

  @@unique([planId, slug])
  @@map("thematic_units")
}

model KnowledgeObject {
  id      String @id @default(uuid())
  planId  String
  thematicUnitId String
  plan    CurriculumPlan @relation(fields: [planId], references: [id])
  thematicUnit ThematicUnit @relation(fields: [thematicUnitId], references: [id])
  name      String
  slug      String
  position  Int
  items     CurriculumItem[]

  @@unique([thematicUnitId, slug])
  @@map("knowledge_objects")
}

model Skill {
  id      String @id @default(uuid())
  subjectId String?
  subject  Subject? @relation(fields: [subjectId], references: [id])
  code     String @unique
  text     String @db.Text
  items    CurriculumItemSkill[]

  @@index([subjectId])
  @@map("skills")
}

model CurriculumItem {
  id      String @id @default(uuid())
  planId  String
  thematicUnitId String
  knowledgeObjectId String
  plan    CurriculumPlan @relation(fields: [planId], references: [id])
  thematicUnit ThematicUnit @relation(fields: [thematicUnitId], references: [id])
  knowledgeObject KnowledgeObject @relation(fields: [knowledgeObjectId], references: [id])
  position  Int
  sourcePage Int?
  notes     String? @db.Text
  terms     CurriculumItemTerm[]
  skills    CurriculumItemSkill[]

  @@index([planId, position])
  @@map("curriculum_items")
}

```

```
}

model CurriculumItemTerm {
  curriculumItemId String
  termId           String
  curriculumItem   CurriculumItem @relation(fields: [curriculumItemId], references: [id],
    onDelete: Cascade)
  term             Term           @relation(fields: [termId], references: [id])

  @@id([curriculumItemId, termId])
  @@map("curriculum_item_terms")
}

model CurriculumItemSkill {
  curriculumItemId String
  skillId           String
  sortOrder         Int           @default(1)
  curriculumItem   CurriculumItem @relation(fields: [curriculumItemId], references: [id],
    onDelete: Cascade)
  skill            Skill          @relation(fields: [skillId], references: [id])

  @@id([curriculumItemId, skillId])
  @@map("curriculum_item_skills")
}
```

4. Estrutura do módulo NestJS

O backend pode iniciar com um módulo de leitura chamado curriculo. Depois, se houver painel administrativo, é possível adicionar rotas de criação, edição e importação protegidas por autenticação.

```
src/
├── app.module.ts
├── prisma/
│   ├── prisma.module.ts
│   └── prisma.service.ts
├── curriculo/
│   ├── curriculo.module.ts
│   ├── curriculo.controller.ts
│   ├── curriculo.service.ts
│   └── dto/
│       └── curriculo-query.dto.ts
```

Responsabilidades por arquivo

Arquivo	Responsabilidade
prisma.service.ts	Abrir e encerrar conexão com o banco via PrismaClient.
prisma.module.ts	Exportar PrismaService para outros módulos.
curriculo.module.ts	Agrupar controller e service do domínio curricular.
curriculo.controller.ts	Receber requisições HTTP e repassar filtros para o service.
curriculo.service.ts	Montar queries Prisma, aplicar filtros e formatar resposta.
curriculo-query.dto.ts	Validar query params como ano, trimestre, componente e habilidade.

DTO de consulta

```
export class CurriculoQueryDto {
  ano?: number;
  area?: string;
  componente?: string;
  trimestre?: number;
  habilidade?: string;
  q?: string;
}
```

Regras de filtros

- ano filtra SchoolYear.gradeNumber.
- area filtra KnowledgeArea.slug.
- componente filtra Subject.slug.
- trimestre filtra Term.number via tabela CurriculumItemTerm.
- habilidade filtra Skill.code via tabela CurriculumItemSkill.
- q faz busca textual em unidade temática, objeto de conhecimento, código ou texto da habilidade.

5. Contrato da API

Método	Rota	Finalidade
GET	/curriculo	Listar itens curriculares com filtros.
GET	/curriculo/:id	Buscar um item curricular específico.
GET	/curriculo/habilidades/:codigo	Buscar itens relacionados a uma habilidade.
GET	/curriculo/planos	Listar planos por ano, área e componente.
GET	/curriculo/anos	Listar anos escolares disponíveis.
GET	/curriculo/componentes	Listar áreas e componentes disponíveis.

Exemplos de chamadas

```
GET /curriculo?ano=1&area=linguagens&componente=lingua-portuguesa
GET /curriculo?trimestre=1&habilidade=EF15LP01
GET /curriculo?q=estrategia%20de%20leitura
GET /curriculo/habilidades/EF15LP02
```

Formato de resposta sugerido

```
{
  "data": [
    {
      "id": "uuid",
      "ano": {
        "codigo": "EF_01",
        "rotulo": "1º ano",
        "numero": 1
      },
      "area": {
        "nome": "Linguagens",
        "slug": "linguagens"
      },
      "componente": {
        "nome": "Língua Portuguesa",
        "slug": "lingua-portuguesa"
      },
      "trimestres": [
        { "numero": 1, "rotulo": "1º trimestre" },
        { "numero": 2, "rotulo": "2º trimestre" },
        { "numero": 3, "rotulo": "3º trimestre" }
      ],
      "unidadeTemática": "Leitura/escuta compartilhada e autônoma",
      "objetoConhecimento": "Estratégia de leitura",
      "habilidades": [
        {
          "codigo": "EF15LP02",
          "texto": "Texto integral da habilidade."
        }
      ],
      "paginaFonte": 321
    }
  ]
}
```

6. Plano de implementação

Etapa	Entrega	Critério de aceite
1. Projeto base	Criar projeto NestJS, configurar PostgreSQL e Prisma.	Servidor sobe localmente e conecta ao banco.
2. Schema e migration	Criar schema.prisma e rodar primeira migration.	Tabelas criadas sem erro no PostgreSQL.
3. Seed inicial	Popular documento, ano, área, componente, trimestres e alguns itens da página 321.	GET /curriculo retorna registros reais.
4. Módulo curriculo	Criar controller, service e DTOs.	Filtros por ano, componente, trimestre e habilidade funcionam.
5. Importador JSON	Ler arquivo curriculo.seed.json e gravar dados normalizados.	Importador evita duplicidade e registra página/ordem.
6. Documentação	Adicionar Swagger/OpenAPI.	Rotas testáveis pela interface de documentação.
7. Revisão de dados	Conferir amostra importada contra o PDF original.	Dados batem com a tabela de origem.

Comandos iniciais

```
npm install @prisma/client class-validator class-transformer
npm install -D prisma
npx prisma init
npx prisma migrate dev -n create_curriculum_model
npx prisma generate
```

Checklist técnico

- Configurar DATABASE_URL no arquivo .env.
- Ativar ValidationPipe global com whitelist e transform.
- Criar índices para filtros frequentes: ano, componente, habilidade e trimestre.
- Adicionar paginação em GET /curriculo quando o documento completo for importado.
- Criar testes para filtros combinados.

7. Estratégia de importação dos dados

A maior dificuldade está na transformação da tabela original. O ideal é não importar diretamente para o banco. Primeiro, deve-se produzir um JSON intermediário revisável, e só depois executar o seed/importador.

```

PDF / imagem / tabela
↓
Extração para JSON bruto
↓
Normalização
↓
Validação humana de amostra
↓
Importador Prisma
↓
PostgreSQL
↓
API NestJS

```

Formato do arquivo de importação

```

[
  {
    "ano": 1,
    "area": "Linguagens",
    "componente": "Língua Portuguesa",
    "trimestres": [1, 2, 3],
    "unidadeTematica": "Leitura/escuta compartilhada e autônoma",
    "objetoConhecimento": "Reconstrução das condições de produção e recepção de textos",
    "habilidades": [
      {
        "codigo": "EF15LP01",
        "texto": "Identificar a função social de textos que circulam em campos da vida social."
      }
    ]
  },
  "paginaFonte": 321,
  "ordem": 1
}
]

```

Validações recomendadas no importador

- Não aceitar item sem ano, área, componente, unidade temática ou objeto de conhecimento.
- Normalizar códigos de habilidade para caixa alta.
- Gerar slug sem acento e sem espaços.
- Usar upsert para não duplicar área, componente, ano, trimestre e habilidade.
- Registrar página de origem e ordem para conferência posterior.

8. Trechos de implementação sugeridos

PrismaService

```
import { Injectable, OnModuleDestroy, OnModuleInit } from '@nestjs/common';
import { PrismaClient } from '@prisma/client';

@Injectable()
export class PrismaService
  extends PrismaClient
  implements OnModuleInit, OnModuleDestroy
{
  async onModuleInit() {
    await this.$connect();
  }

  async onModuleDestroy() {
    await this.$disconnect();
  }
}
```

Controller principal

```
@Controller('curriculo')
export class CurriculoController {
  constructor(private readonly service: CurriculoService) {}

  @Get()
  findAll(@Query() query: CurriculoQueryDto) {
    return this.service.findAll(query);
  }

  @Get('planos')
  findPlans() {
    return this.service.findPlans();
  }

  @Get('habilidades/:codigo')
  findBySkillCode(@Param('codigo') codigo: string) {
    return this.service.findBySkillCode(codigo);
  }

  @Get('/:id')
  findOne(@Param('id') id: string) {
    return this.service.findOne(id);
  }
}
```

Consulta Prisma com filtros

```
const items = await prisma.curriculumItem.findMany({
  where: {
    plan: {
      schoolYear: { gradeNumber: 1 },
      subject: {
        slug: 'lingua-portuguesa',
        area: { slug: 'linguagens' },
      },
    },
    terms: { some: { term: { number: 1 } } },
    skills: { some: { skill: { code: 'EF15LP02' } } },
  },
  include: {
    plan: { include: { schoolYear: true, subject: { include: { area: true } } } },
    thematicUnit: true,
    knowledgeObject: true,
    terms: { include: { term: true } },
    skills: { include: { skill: true } },
  },
});
```

9. Roadmap sugerido

Semana	Foco	Resultado esperado
1	Base do projeto e banco	NestJS, Prisma e PostgreSQL configurados; migrations criadas.
2	Módulo currículo	Rotas GET e filtros básicos funcionando com seed manual.
3	Importador JSON	Importação da página 321 e validação de dados.
4	Expansão do documento	Importação por blocos: mais páginas, anos e componentes.
5	Documentação e testes	Swagger, testes de filtros e revisão de performance.

Riscos e mitigação

Risco	Impacto	Mitigação
PDF com células mescladas	Dados podem ficar deslocados.	Normalizar em JSON e revisar amostras.
Habilidades repetidas	Duplicidade no banco.	Usar Skill.code como chave única.
Texto longo de habilidade	Resposta da API pode ficar grande.	Permitir paginação e seleção de campos no futuro.
Mudança de versão do currículo	Dados antigos podem se misturar.	Manter CurriculumDocument.version e sourceUrl.
Importação incompleta	Frontend exibe lacunas.	Registrar status por página/bloco importado.

Critérios de aceite do MVP

- GET /curriculo retorna os itens importados da página de referência.
- Filtros por ano, área, componente e trimestre podem ser usados juntos.
- Busca por código de habilidade retorna todos os itens relacionados.
- Cada item informa unidade temática, objeto de conhecimento, habilidades e página fonte.
- Banco suporta a importação de outros anos e componentes sem remodelagem estrutural.

10. Próximos passos práticos

Para iniciar o desenvolvimento, a ordem mais segura é criar primeiro a modelagem e o módulo de consulta, depois importar uma amostra pequena da tabela e validar a resposta da API. Depois que o formato estiver aprovado, a extração do restante do documento pode ser feita em lotes.

1. Criar repositório do backend NestJS.
2. Adicionar Prisma e PostgreSQL.
3. Aplicar o schema inicial deste plano.
4. Criar seed com a página 321.
5. Subir GET /curriculo e testar no Postman/Insomnia ou Swagger.
6. Definir o formato final do JSON intermediário para importação do PDF completo.

Entrega final do MVP

Backend NestJS consultando PostgreSQL e entregando dados curriculares por API REST, preparado para integração com um frontend React/Vite ou outro cliente web.